Estimating stochastic actor-oriented network models in Stata

Thomas U. Grund
Institute of Sociology, RWTH Aachen University, Germany

Abstract. `nwsaom` fits stochastic actor-oriented models (Siena models) to panel network data in Stata. It uses the same method-of-moments algorithm and the same effects as RSiena, entirely within Stata, and implements 1,058 of the 1,201 effects in the effects table of RSiena 1.6.6. Beyond one network, it handles undirected and two-mode networks, several networks and behaviors, and network-behavior co-evolution; it also offers maximum-likelihood and Bayesian estimation, goodness of fit, simulation, score tests, joint estimation of several groups, and meta-analysis across groups. The simulation runs in a compiled, multithreaded plugin. On 27 benchmark models the estimates agree with RSiena's to within 0.10 standard errors; the time per fit on a single thread is lower by a factor of 1.9 to 11.5 (median 3.8). The article describes the syntax, the effect library, the estimation, and the diagnostics, with a published adolescent-friendship panel as the running example.

Keywords: st0004, nwsaom, stochastic actor-oriented models, SAOM, Siena models, network dynamics, method of moments, Stata plugins, nwcommands

1 Introduction

Network data collected to test theories of tie formation are often a short panel: the same actors observed at two or more points in time. A stochastic actor-oriented model (SAOM) (Snijders, 1996, 2001; Snijders et al., 2010) is a statistical model built for exactly that situation. It explains how a network changed between the waves and, when the actors also have a characteristic that changes, such as smoking, how the network and the characteristic shaped each other. Section 3 explains the idea in plain words.

The `RSiena` package for R (Ripley et al., 2024) is the established implementation of these models and has been developed over roughly two decades. Researchers who work in Stata have so far fitted a SAOM by exporting the panel network to R, running RSiena there, and bringing the results back.

`nwsaom` makes the same models available within Stata. It builds on the network objects and commands of `nwcommands` (Grund, 2026a), fits the same effects with the same three-phase method-of-moments algorithm, supports conditional and unconditional estimation and joint network-behavior co-evolution (Huisman and Snijders, 2003), and gives estimates that agree with RSiena's (Section 7.2). The simulations run in a compiled plugin that uses all processor cores; on a single thread the benchmark of Section 7.2 finds the time per fit lower by a factor of 3.5 to 4.3 for the models with `gwesp()` and of 3.5 to 11.5 without it. Of the 1,201 effects in the effects table of RSiena 1.6.6 (as `getEffects()` returns it for twelve data configurations), `nwsaom` implements 1,058, each compared with `siena07` (three by the observed statistic only; Section 6.3 says what is missing). `nwsaom` is meant for researchers who work in Stata or prefer it to R; it is not

a replacement for RSiena, which remains the reference implementation and the more complete program.

A manual of the `nwsaom` command, with all effects, the comparison with RSiena and further examples, is archived at Zenodo (Grund, 2026) (`https://doi.org/10.5281/zenodo.23623807`); the package as a whole is described in Grund (2026a), and the network analysis with Stata, including a chapter on stochastic actor-oriented models, is the subject of a forthcoming book (Grund, 2027). Further model packages are described on `https://nwcommands.org`.

2 Installation

`nwsaom` is a package of its own. It uses the network objects, data handling and plotting commands of `nwcommands` (Grund, 2026a), which must be installed first; installation is repeated here in full so this article stands on its own. Stata 17 or later is needed (the software is tested with Stata 17, 18 and 19; StataBE is enough). From within Stata:

```
. net install nwcommands, from(https://nwcommands.org/net/) replace
. nwinstall, everything
```

The first command installs `nwinstall`. The second installs the `nwcommands` core (the full command set, the help files and the dialog boxes), the four model packages including `nwsaom`, and a "Network Analysis" menu in Stata, in one step, and prints the installed versions at the end. Only the core and `nwsaom` are needed for this article: `nwinstall, all` followed by `nwinstall, modelpkg(nwsaom)` installs just those two (add `nogui` to skip the menu). Repeating the lines updates the installation. No compiler is needed on the user's own machine. The native backend (Section 7.1) ships as precompiled plugins for macOS, Linux and Windows; the Linux and Windows builds are produced by the package's continuous integration. The results in this article were produced on macOS; its examples give the same numbers on Windows, where the package's full test suite also passes. The more recent effects exist only in the plugin, and `nwsaom` stops with a message that says so when the plugin is unavailable; the older effects have a Mata implementation of the simulation that gives identical results but is much slower, and a fit that falls back to it prints one warning line that says why (plugin file not found, plugin too old, model beyond the plugin's capacity); `quietly` suppresses it. The manual (Grund, 2026) documents every option; it is archived at Zenodo under the DOI of all versions, `https://doi.org/10.5281/zenodo.23623807`. The source code of all packages is public at `https://github.com/thomas-u-grund`, with `nwsaom` at `https://github.com/thomas-u-grund/nwsaom` and the release this article describes at `https://github.com/thomas-u-grund/nwsaom/releases/tag/v1.5.8`. The C source of the plugin is part of the package, together with a note on how to build it; the compiled plugins are what `net install` installs, and the source is copied into the current directory by

```
. net get nwsaom, from(https://raw.githubusercontent.com/thomas-u-grund/nwsaom/main)
```

The source also serves as a fallback. If the plugin of a platform is missing, is older than the `.ado` files (the Linux and Windows files are built by continuous integration and can

lag behind the macOS file), or does not load on a particular system, the plugin can be compiled on the computer that needs it:

```
. nwinstall, buildplugin(nwsaom)
```

The command fetches the source from the repository, compiles it with the local C compiler and installs the plugin where `net install` puts it, keeping an existing file with the extension `.prev`; a C compiler and `make` are required (on macOS the command line tools of Xcode), and Stata has to be restarted afterwards. A plugin does not depend on the version or the flavor of Stata (BE, SE, MP) but on the operating system and the processor. The routine was tested on macOS; on Windows it stops with a message, and the build note that comes with the source gives the command for the Visual Studio command line.

Versions

This article describes `nwsaom` version 1.5.8 (9 November 2026) with `nwcommands` version 3.17.2; it needs `nwcommands` version 3.6.0 or later. The listings were produced with these versions in Stata 17, and Stata 18 and 19 give the same results. Releases follow the usual major.minor.patch scheme: a change in the first number may change results or syntax, the second adds features, and the third fixes errors. Each release is a tagged version on GitHub, and the tag can be given in place of `main` to install exactly that release:

```
. local url "https://raw.githubusercontent.com/thomas-u-grund/nwsaom/v1.5.8/"
. net install nwsaom, from("`url'") replace
```

`nwsaom version` reports the installed release, the version of the `nwcommands` core, the plugin in use and the version of RSiena that the results were compared with; `e(version)` stores the release after each fit, so a published analysis can state which release produced it. `nwversion` lists the core and all installed model packages.

Licence

`nwsaom` is free software under the GNU General Public License, version 3, as are the `nwcommands` core and the other model packages. `nwsaom` is derived from RSiena (Ripley et al., 2024), which is dual-licensed (GPL-2 or GPL-3, at the user's option); `nwsaom` uses the GPL-3 option. For Bayesian estimation it follows the algorithm of the multiSiena package (Snijders et al., 2024) (GPL-3). Further worked tutorials are at `https://nwcommands.org/nwsaom/`.

3 SAOM framework

A SAOM does not describe an observed network so much as the process assumed to have produced its change (Snijders, 1996, 2001; Snijders et al., 2010; Snijders, 2017). Suppose the same pupils name their friends in three annual surveys. Between two surveys many friendships appear and end, but nobody sees it happen. The model fills the gap with a chain of small steps, called *ministeps*: at moments set by a *rate function*, one actor i is

chosen and may create or drop a single one of its own outgoing ties – never anyone else's – or leave things as they are. Many such steps carry the first network to the second. This is a modelling device, not a claim about how adolescents really decide; it lets us ask which tendencies, on average, would reproduce the change we observe.

Two sets of numbers describe the process, and they answer different questions. The *rate* says how often an actor gets a turn: a rate of 6 for a period means that each pupil had on average about six chances to change a friendship. The *effects* say what an actor does with a turn. Once activated, actor i weighs its options – one for each tie it could create or drop, plus doing nothing – through an *evaluation function*, a weighted sum of effects:

$$f_i(\theta, x) = \sum_k \theta_k \, s_{ki}(x). \tag{1}$$

Here $s_{ki}(x)$ is the value of effect k for actor i in the network x that the option would produce (for example, the number of i's ties that are reciprocated), and θ_k is the coefficient to be estimated. Actor i then moves to x' with a multinomial (conditional) logit probability over its C available options (McFadden, 1973):

$$P(x \to x') = \frac{\exp\{f_i(\theta, x')\}}{\sum_{x'' \in C} \exp\{f_i(\theta, x'')\}}. \tag{2}$$

Options with a higher value are chosen more often but never with certainty, hence *stochastic*. The actor looks only at how an option changes its own position – its own ties and the ties around them – and never at what it would mean for anyone else; this is what *actor-oriented* and the model's myopia refer to. A positive coefficient means that options with the feature become more likely, a negative one that they become less likely, and the effects are adjusted for one another, so the table reads much like a logistic regression.

Under the default, a constant rate, every actor is equally likely to be next: the time until the next ministep, pooled across all n actors, is exponential with rate $n \times$ rate, and whichever actor acts is drawn uniformly. `ratecov()` (Table 1) lets the rate vary with a covariate, $\text{rate}_i = \text{rate}_0 \exp(\beta x_i)$. The rates are always reported apart from the effect table.

Adding a behavior variable, such as smoking, that also changes over time opens a second question. The behavior then changes in small steps too, and a joint fit can estimate a network-side effect (does similarity on the behavior predict which new ties form – *selection*) and a behavior-side effect (does an actor's own value drift toward its alters' average – *influence*). A cross-sectional design cannot separate the two, because both leave the same trace of friends who resemble each other; a model of change can, because it asks which came first (Steglich et al., 2010; Kalish, 2020). Section 5.5 works through this on the Glasgow panel.

3.1 Estimation and convergence

A SAOM cannot be fitted by an ordinary formula, because its likelihood sums over every ministep sequence that could have produced the observed waves. By default `nwsaom`

therefore uses the *method of moments*, as RSiena does: it simulates the process under trial coefficients and adjusts them until the simulated networks resemble the observed ones on average. This takes a few seconds to minutes, differs slightly from run to run (set `seed()` to reproduce a fit), and ends with a check of whether the simulated and observed statistics agree, the *convergence* check that Section 5.3 shows how to read. Maximum-likelihood (`ml`) and Bayesian (`nwsaom bayes`) estimation are also available for a single directed network; they are slower, and the method of moments remains the default. The algorithm, its options and the comparison with RSiena's maximum-likelihood and Bayesian estimators are in Appendix A.

4 The nwsaom command

4.1 Basic syntax

. `nwsaom , waves(`*netlist*`) outdegree` [*effect* ...] [*control options*]

A model is written as a plain list of effect options, the way any Stata regression command lists covariates: `outdegree` is required (it sets the overall tendency to have ties), and any number of further effects can follow. For an undirected network the same effect is the actor's degree, the number of its ties, and it is written `degree` (RSiena calls it "degree (density)"; `density` is accepted too, and `outdegree` keeps working). A tie counts once for each of its two actors; the command and the effect list are otherwise the same. `nwsaom` recognizes an undirected network from `nwset, undirected` or from waves that are symmetric, and then lets two actors agree on a tie in the way chosen by `symtype()` (below); effects that only make sense for directed ties, such as `reciprocity`, are not available for it. The observed waves go in `waves()`, a list of networks (a *netlist* of `nwcommands`, so that patterns such as `glasgow*` or `w1-w3` work, expanded in the order in which the networks were created), two or more of them in time order; it strings every consecutive pair into its own inter-wave period and pools the effect coefficients across all of them into one method-of-moments fit, as in RSiena, while the rate parameter is still estimated separately for each period (`e(rates)`/`e(rate_tratios)`), since how often actors get a turn need not stay constant across a panel even when what they do with a turn does.

What the networks must look like. As in RSiena, the ties of every wave are binary. A wave that holds any tie value other than 0 and 1 (shares, counts or other real values) is refused with an error, whether or not the network was declared valued with `nwset`; such a network is dichotomized first with `nwdichotomize`. Dyads whose value is fixed by design, which RSiena codes as 10 and 11 in the network array, are marked instead with a separate 0/1 matrix in `structural()`. All waves must list the same actors in the same order.

4.2 Effect library

Table 1 sets out `nwsaom`'s current effect library, organized by family and by which evaluation function each effect belongs to. Rows above the rate-function entry belong to the network's evaluation function; rows below it belong to the behavior's, and

require `behavior()` (Section 4.3). Several node-covariate options double as RSiena aliases alongside the names used elsewhere in `nwcommands`: `samex()` for `nodematch()`, `altx()` for `nodeicov()`, `egox()` for `nodeocov()`, and `simx()` for `simcov()` – so a reader arriving from RSiena's own effects table does not have to relearn a new vocabulary. Every node-covariate option takes a varlist and adds one effect per variable, centered as RSiena centers covariates; `e(rsiena_labels)` holds RSiena's name for each effect, and the option `detail` lists them below the table. `interact()` adds two- and three-way interactions of effects already in the model, following RSiena's `includeInteraction()` rules.

Table 1: `nwsaom`'s effect library, by family and evaluation role.

Effect	Role	Description
Baseline (one required)		
`outdegree`	network	Tie-creation propensity (density/intercept)
Reciprocity		
`reciprocity`	network	Reciprocated-tie propensity
`reciprocityendow` / `reciprocitycreation`	network	Split into loss- and gain-side coefficients
Node covariates (one varname each)		
`nodematch()`	network	Homophily on a categorical attribute (alias `samex()`)
`nodecov()`	network	Sum of sender's and receiver's covariate value
`nodeicov()`	network	Receiver (in-)covariate effect (alias `altx()`)
`nodeocov()`	network	Sender (out-)covariate effect (alias `egox()`)
`simcov()`	network	Covariate similarity (alias `simx()`)
Degree-based structural		
`indegpopularity`	network	Preferential attachment to high in-degree alters (sqrt)
`outpopularity`	network	Preferential attachment to high out-degree alters (sqrt)
`outactivity`	network	Sender's own out-degree activity (squared)
`inactivity`	network	Sender's own in-degree activity (sqrt)
Triadic closure		
`transtrip`	network	Transitive triplets ($i \to k \to j$, $i \to j$)
`transmedtrip`	network	Transitive mediated triplets
`transties`	network	Existence-indicator transitive closure
`transrectrip`	network	Transitive reciprocated triplets
`cycle3`	network	Directed 3-cycles
`cycle4`	network	Directed 4-cycles (base parameterization)
`gwesp()`	network	Geometrically weighted edgewise shared partners
Isolates and balance		
`balance`	network	Structural balance (data-derived constant, no user parameter)
`isolatenet` / `isolatepop`	network	Isolate avoidance / isolate popularity
`outiso` / `antiiso`	network	Out-isolate count / anti-isolation
`antiiniso` / `antiiniso2`	network	In-isolation avoidance (two parameterizations)
Degree assortativity		

continued on next page

Table 1 (continued)

Effect	Role	Description
`outoutass` / `ininass`	network	Out-out / in-in degree assortativity
`outinass` / `inoutass`	network	Out-in / in-out degree assortativity
Rate function		
`ratecov()`	rate	Covariate-dependent activation rate, $\text{rate}_i = \text{rate}_0 e^{\beta x_i}$
Behavior co-evolution (require `behavior()`)		
`linear`	behavior	Tendency toward higher/lower values (required baseline)
`quadratic`	behavior	Tendency toward extreme values (self-reinforcement)
`quadraticendow` / `quadraticcreation`	behavior	Split into decrease- and increase-side coefficients
`avalt`	behavior	Assimilation toward network alters' average value (influence)
`avaltendow` / `avaltcreation`	behavior	Split into decrease- and increase-side coefficients
`avsim`	behavior	Tendency to maximize similarity to alters (influence)
`avsimendow` / `avsimcreation`	behavior	Split into decrease- and increase-side coefficients

The table shows the main families; the manual (Grund, 2026) lists all 1,058 implemented effects with their RSiena names, and Section 6.3 says what is not implemented.

Snijders et al. (2010) and Snijders (2017) cover what each family means substantively – when a finding is better expressed through `transtrip` than `cycle3`, say, or how isolate-avoidance behaves differently on a sparse network than a dense one – well beyond what a syntax table like this one can show. Effects in the isolate and assortativity families can make estimation unstable on small or sparse networks, in `nwsaom` as in RSiena: a single tie change can alter other actors' isolate status or degrees, and such networks contain few of the configurations these effects describe.

4.3 Co-evolution, composition change, and undirected networks

`behavior(`*varlist*`)` – one variable per wave – switches `nwsaom` into co-evolution mode, where `linear` becomes a required baseline (the behavior-side counterpart to `outdegree`) and `quadratic`, `avalt`, and `avsim` are available as optional behavior effects, all estimated together with the network-side effects in one method-of-moments fit (Section 5.5). Several behaviors can be listed in one call (`behavior(a1 a2 a3 : linear avalt | s1 s2 s3 : linear)`). `present()`, `joinat()`/`leaveat()`, `missnet()` and `missbeh()` cover composition change (actors joining or leaving between waves or, with `joinat()` and `leaveat()`, at any time within a period; RSiena's `sienaCompositionChange()`) and missing tie or behavior data; with `present()` an actor can act in a period if it is present at the wave at which the period starts, as in RSiena; `groups()` fits one model to several groups of actors at once (Section 5.9); `structural()` marks dyads whose value is fixed by design (two waves, network only). `symmetric` moves to a mutual-consent ministep for a network whose ties are already symmetric at both waves; `symtype()` chooses among RSiena's model types for a non-directed relation: `forcing`, the default

as in RSiena, lets the actor who chooses a tie impose it; `confirmation` needs the other actor to accept a new tie; `force`, `agree` and `joint` are RSiena's pairwise model types, in which a pair of actors decides together. Table 2 lays out these and `nwsaom`'s remaining controls, grouped as in `help nwsaom`.

Table 2: `nwsaom` control options, by group.

Option	Description
Observed waves (exactly one required)	
`wave1(netname) wave2(netname)`	Exactly two panel waves
`waves(namelist)`	Two or more waves, chained into $n-1$ pooled periods
Co-evolution	
`behavior(varlist)`	One behavior variable per wave; enables joint network+behavior estimation
`behtheta0(numlist)`	Starting values for behavior-side coefficients; default 0
Composition change and missing data	
`present(varlist)`	One 0/1 joiner/leaver indicator per wave
`missnet(matlist)`	One 0/1 missing-tie indicator matrix per wave
`missbeh(varlist)`	One 0/1 missing-behavior indicator per wave (requires `behavior()`)
Undirected networks	
`symmetric`	Non-directed relation (automatic for undirected or tie-symmetric waves; any number of waves)
`symtype(string)`	RSiena's model type: `forcing` (default), `confirmation`, `force`, `agree`, `joint`
Covariates, rates, and constraints	
`nocenter`	Use raw covariate values (default: centered by their means, as RSiena)
`uncentered(varlist)`	Use raw values for the listed covariates only
`ratecov(varlist)`	Covariates that change how often actors may change ties
`structural(matname)`	0/1 matrix of dyads that cannot change (structural zeros/ones)
Estimation control	
`unconditional`	Unconditional estimation (default for network-only models: conditional, as RSiena)
`theta0(numlist)`	Starting values, network-side effects; default 0
`rate0(numlist)`	Starting values of the rates (unconditional estimation)
`k0(int)`	Phase 1 replicate count; default 50
`k3(int)`	Phase 3 replicate count; default 1,000
`firstg(real)`	Phase 2 starting Robbins-Monro gain; default 0.2
`seed(int)`	Random-number seed
`cores(int)`	Number of threads; default all physical cores (results do not depend on it)
Estimator	
`ml`	Maximum likelihood instead of the method of moments (single directed network, with or without one behavior)
`mloptions(string)`	Controls of the maximum-likelihood estimator (`n1()`, `nsub()`, `n3()`, `gain()`, ...)
Reporting	

continued on next page

Table 2 (continued)

Option	Description
`detail`	Also list RSiena's effect names, covariate means, and every convergence t-ratio
Postestimation (`estat gof`)	
`nsim(int)`	Simulated replicates at fitted coefficients; default 50
`stats(namelist)`	Auxiliary statistics to check: `outdegree indegree geodesic triad behavior`
`maxdeg(int)` / `maxdist(int)`	Truncate the degree / geodesic-distance distribution; default 15 / 6
`twotailed`	Two-tailed rather than the default one-tailed test
`join(off)`	Test each period separately rather than pooled

5 Remarks and examples

The examples that follow use the Glasgow data: a 50-pupil excerpt of the Teenage Friends and Lifestyle Study (Michell and West, 1996; Pearson et al., 2006), three annual waves of directed friendship nominations among Scottish schoolgirls, with smoking, alcohol, and sport attributes recorded at each wave. It is the same panel behind RSiena's `s50` tutorial data, and the source of every benchmark reported in Section 7. It ships with `nwcommands` and loads directly with `nwwebuse glasgow`.

5.1 Looking at the data first

Figure 1 draws the three waves of the friendship network. Each dot is a pupil, and an arrow from one pupil to another means that the first named the second as a friend in that wave. Every pupil has the same position in all three panels, so the pupils do not move from panel to panel: what changes is the set of arrows. Pupils who are close together are linked by friendships, so the clusters in the plot are groups of friends; the shade and shape of a dot show the pupil's smoking at the first wave (nonsmoker, occasional smoker, regular smoker). Look for the arrows that appear or disappear between waves while the groups stay broadly in place, and for the two pupils at the right who name nobody and are named by nobody in any wave.

The figure is drawn with `nwplot`. The layout is computed once from wave 2 and reused for the other waves, so that every pupil keeps the same position:

```
. nwwebuse glasgow, nwclear
. nwplot glasgow2, layout(kk) generate(gx gy) nodraw
. local opt color(smoke1, colorpalette(gs11 gs6 black))
>     symbol(smoke1, legendoff symbolpalette(circle diamond square))
>     nodefactor(1) arrowfactor(.7) legend(rows(1) size(small))
. nwplot glasgow1, nodexy(gx gy) `opt' title("Wave 1") name(w1, replace)
. nwplot glasgow2, nodexy(gx gy) `opt' title("Wave 2") name(w2, replace)
. nwplot glasgow3, nodexy(gx gy) `opt' title("Wave 3") name(w3, replace)
. graph combine w1 w2 w3, rows(1)
```

Before a fit, `nwsaom describe` prints what RSiena prints in its data report: for every wave the number of ties, the density and the average degree, and for every period the numbers of ties kept, created and dissolved, the Jaccard index (kept ties as a share of all ties present at either wave) and the Hamming distance. With `behavior()` it also gives

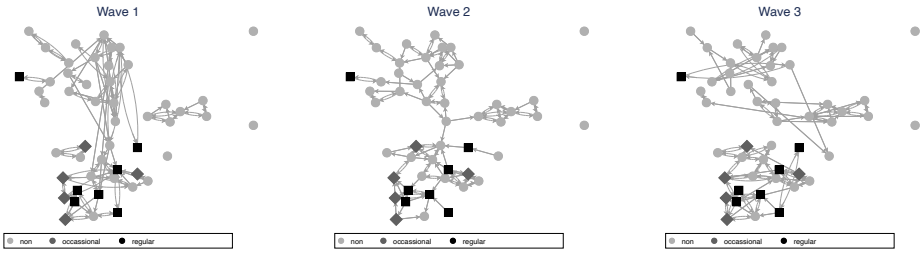


Figure 1: The Glasgow friendship network at waves 1, 2 and 3 (50 pupils; an arrow points from the pupil who names a friend to the pupil named). The positions are the same in all panels (Kamada-Kawai layout of wave 2). Shade and shape: smoking at wave 1.

the behavior's means and how many pupils move up, down or stay. On the Glasgow data:

```
. nwsaom describe, waves(glasgow1 glasgow2 glasgow3)

Network data: 3 waves, 50 actors, directed
-----------------------------------------------------
        Wave |     ties     density   avg. degree
-------------+---------------------------------------
           1 |      113      0.0461         2.260
           2 |      116      0.0473         2.320
           3 |      122      0.0498         2.440
-----------------------------------------------------

-------------------------------------------------------------------
      Period |    kept   created  dissolved   Jaccard   Hamming
-------------+-----------------------------------------------------
        1->2 |      57        59         56     0.331       115
        2->3 |      66        56         50     0.384       106
-------------------------------------------------------------------
```

About a third of the ties of one wave are still there at the next. The usual rule of thumb in the SAOM literature is that a Jaccard index below about 0.3 means that too few ties are stable for the data to say much about the process; `nwsaom describe` warns when a period falls below it.

5.2 A first fit

`outdegree` on its own – tie-formation propensity, nothing else – would be the SAOM equivalent of an intercept-only model. The example below goes further, asking whether smoking status predicts new friendship formation once reciprocity and cyclical triadic closure are accounted for, chaining all three Glasgow waves into a single pooled fit

through `waves()`. Figure 2, below the listing, shows what each of the four effects counts.

```
. nwwebuse glasgow, nwclear
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity cycle3 nodemat
> ch(smoke1) k0(50) k3(2000) seed(12345)
SAOM (Method of Moments, conditional), waves: glasgow1 glasgow2 glasgow3
Actors: 50                              Periods: 2
```

glasgow1_to~3	Coefficient	Std. err.	z	P>|z|	[95% conf.	interval]
outdegree	-2.806135	.1620405	-17.32	0.000	-3.123729	-2.488542
reciprocity	2.620083	.1870256	14.01	0.000	2.253519	2.986646
nodematch_s~1	.1214596	.1506553	0.81	0.420	-.1738195	.4167387
cycle3	1.161113	.1459085	7.96	0.000	.875137	1.447088

```
Rate parameters (estimated, one per inter-wave period):
```

	period1	period2
rate	6.6140	5.3318
se	1.1772	0.8996

```
Overall maximum convergence ratio: 0.120 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

Reciprocity and cyclical closure (`cycle3`) are both strongly positive, while smoking homophily is not significant (`nodematch_smoke1`, $p = 0.420$). The rate table gives the estimated rate of each period: on average, a pupil had about 6.6 opportunities to change a friendship between the first and second waves and 5.3 between the second and third. The last lines summarize convergence (explained below): the overall maximum convergence ratio is 0.120 and every t-ratio is below 0.1, so the fit meets both of RSiena's criteria. This run used 2,000 replicates in Phase 3 (`k3(2000)`); with fewer replicates a first fit can miss the second criterion by a small margin, and the next section shows what to do then. The estimates, standard errors and rates agree with RSiena's (Section 5.7).

What the last lines mean. A SAOM is fitted by simulation. `nwsaom` starts from trial values of the coefficients, lets the model play out the story above on the computer many times, and compares what the simulated networks look like with what you observed: for example, how many ties are reciprocated at the end of the period. If the simulated networks show too few reciprocated ties, the coefficient for reciprocity is raised; if too many, it is lowered. This is repeated, with ever smaller adjustments, until the simulated and observed values agree on average. At that point the estimates have *converged*.

The output reports how closely they agree. For each parameter there is a *convergence t-ratio*, the average gap between simulated and observed statistic, divided by its standard deviation. A value of 0 means a perfect match. The overall maximum convergence ratio summarizes all of them in one number. The conventional rule, used here as in RSiena, is that every t-ratio should be below 0.1 in absolute value and the overall ratio below 0.25. This is a different question from whether a coefficient is statistically significant, and it comes first: the standard errors and p-values are only meaningful once the fit has converged. If it has not, `nwsaom` says so, and the next section shows what to do.

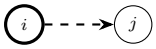
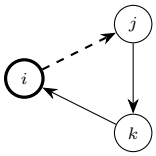
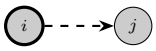
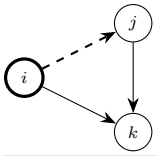
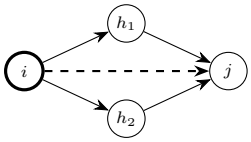
Diagram	Statistic $s_{ki}(x)$	Description
	$\sum_j x_{ij}$	`outdegree`: the number of ties i sends; the baseline tendency to name friends.
	$\sum_j x_{ij}x_{ji}$	`reciprocity`: i names j who already names i.
	$\sum_{j,k} x_{ij}x_{jk}x_{ki}$	`cycle3`: i names j, who names k, who names i (cyclical closure).
	$\sum_j x_{ij}\,[v_i = v_j]$	`nodematch(smoke1)`: i names j with the same smoking level (here shaded alike).
	$\sum_{j,k} x_{ij}x_{jk}x_{ik}$	`transtrip`: i names j and k, and j names k (transitive triplets), each counted in full.
	$\sum_j x_{ij}\, e^{\alpha}\{1-(1-e^{-\alpha})^{p_{ij}}\}$ with $p_{ij} = \sum_h x_{ih}x_{hj}$	`gwesp()`: the shared partners h of the tie $i \to j$, with declining weights, so a tie with many shared partners adds little beyond the first few.

Figure 2: What the effects count. Actor i (thick circle) has a turn and considers the dashed tie, which would be created; solid arrows are ties that already exist. The statistic is what the effect adds up for actor i ($x_{ij} = 1$ if i names j, v_i is i's smoking level, and α is the decay parameter of `gwesp()`). An option that completes the configuration is more attractive when the coefficient is positive. The first four effects are in the first fit; `transtrip` and `gwesp()` appear later.

5.3 Reading the convergence summary

When convergence is not perfect, the summary says which criterion is not met and which parameters need attention. Replacing `cycle3` by transitive triplets (`transtrip`; Figure 2) and shortening Phase 3 to 200 replicates, which makes the convergence statistics noisy, gives a fit that has not converged; the second command restarts from its estimates with `theta0()` and the default length of Phase 3:

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip k0(5
> 0) k3(200) seed(12345)

SAOM (Method of Moments, conditional), waves: glasgow1 glasgow2 glasgow3
Actors: 50                              Periods: 2

------------------------------------------------------------------------------
glasgow1_t~3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |  -2.682831   .1090026   -24.61   0.000    -2.896472    -2.46919
 reciprocity |   2.471578   .2064743    11.97   0.000     2.066895     2.87626
   transtrip |   .6214455   .0701761     8.86   0.000     .4839029      .758988
------------------------------------------------------------------------------

Rate parameters (estimated, one per inter-wave period):

-------------------------------
             |   period1    period2
-------------+-----------------
        rate |     6.5504     5.2469
          se |     1.1470     0.9500
-------------------------------

Overall maximum convergence ratio: 0.222 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for r
> eciprocity (0.126) transtrip (0.120)
Convergence is good only if both criteria are met. Refit starting from these est
> imates with theta0(); see nwsaom.

. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transtrip k0(5
> 0) k3(1000) theta0(-2.683 2.472 0.621) seed(12346)

SAOM (Method of Moments, conditional), waves: glasgow1 glasgow2 glasgow3
Actors: 50                              Periods: 2

------------------------------------------------------------------------------
glasgow1_t~3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |  -2.673817   .1181376   -22.63   0.000    -2.905362   -2.442271
 reciprocity |   2.455474   .1969401    12.47   0.000     2.069478    2.841469
   transtrip |    .616723   .0724545     8.51   0.000     .4747147    .7587312
------------------------------------------------------------------------------

Rate parameters (estimated, one per inter-wave period):

-------------------------------
             |   period1    period2
-------------+-----------------
        rate |     6.4119     5.2439
          se |     1.0716     0.8377
-------------------------------

Overall maximum convergence ratio: 0.084 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

The overall maximum convergence ratio of the first fit (0.222) is below 0.25, but two t-ratios are above 0.1 (reciprocity 0.126, `transtrip` 0.120), so `nwsaom` says that the t-ratio criterion is not met and that convergence is good only if both criteria are. The remedy is the one in the log: run the model again, starting from these estimates with

`theta0()`, which is the counterpart of RSiena's `prevAns` (RSiena skips Phase 1 when the model is unchanged; `nwsaom` repeats it). The second fit has an overall ratio of 0.084 and every t-ratio below 0.1. A longer Phase 3 (`k3()`) gives more precise t-ratios, and the estimates hardly move. If a restart does not help, or if the overall ratio stays above 0.25, `nwsaom` says the fit has not converged. When repeating the run does not help, the usual advice is to simplify the model: add an effect that captures the feature the model misses (for example `gwesp()` for closure), include rate effects, fix a parameter that wanders to a very large value with `fix()` and test it with `estat score`, or drop an effect that is nearly collinear with another; `estat vce, corr` shows the correlations of the estimates. The Wald tests of linear hypotheses about the coefficients are Stata's `test`, `lincom` and `testparm`, which use `e(b)` and `e(V)`.

5.4 Checking fit after estimation

`estat gof`, available after any `nwsaom` fit, draws fresh networks at the fitted coefficients and compares their auxiliary statistics to the observed network with a Mahalanobis-distance test (Lospinoso and Snijders, 2019), the same construction behind RSiena's `sienaGOF()`. Run on the well-behaved model above:

```
. estat gof, seed(12345) name(glasgow_gof_plot)
Goodness of fit: Mahalanobis distance test, 50 simulations at the estimates (2 p
> eriods pooled)
```

Statistic	Mahalanobis	P-value	Fit
Outdegree	12.462	0.160	ok
Indegree	8.184	0.600	ok
Geodesic	34.862	0.000	rejected

```
Fit is rejected when P < 0.05 (one-tailed test).
Violin plots: glasgow_gof_plot_outdegree glasgow_gof_plot_indegree glasgow_gof_p
> lot_geodesic
```

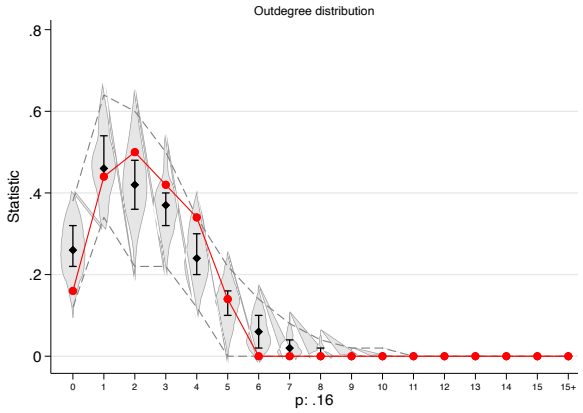


Figure 3: `estat gof`'s outdegree-distribution violin plot for the model of Section 5: simulated distribution (violin), interquartile range (bar), 95% envelope (dashed), and the observed network (red).

Neither the outdegree nor the indegree distribution shows evidence against the fitted model ($p = 0.160$, $p = 0.600$), but the distribution of geodesic distances does ($p < 0.02$; none of the 50 simulations was as far from the center of the simulated distribution as the observed network): the model reproduces who has how many friends, but not how far apart pupils are in the network. With 1,000 simulations in place of 50 the conclusions are the same ($p = 0.535$, 0.587 and 0.001), and `sienaGOF` in RSiena gives $p = 0.600$, 0.689 and < 0.001 for the same model. Effects that shape the network's larger structure, such as `gwesp()`, would be the next step.

5.5 Co-evolution: separating selection from influence

The model below carries the same three Glasgow waves into a full co-evolution fit. `behsim` asks whether similarity in alcohol use, as it currently is, predicts which new friendships form – selection, the network-side counterpart to `nodematch(smoke1)` above (RSiena's `simX` with the behavior as the interacting variable) – while `avalt` (Figure 4), with the behavior given as `behavior(alcohol1 alcohol2 alcohol3)`, asks whether an actor's own alcohol use drifts toward its friends' average – influence. Both are estimated together, in one fit. Alcohol stands in for smoking here because smoking changes little between waves (76% of pupils are non-smokers at the first wave), which leaves too little information to estimate influence:

```
. nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity behsim behavio
> r(alcohol1 alcohol2 alcohol3) linear avalt k0(50) k3(2000) seed(12345)

SAOM co-evolution (Method of Moments), waves: glasgow1 glasgow2 glasgow3
Actors: 50                              Periods: 2
Behavior: alcohol1 alcohol2 alcohol3

------------------------------------------------------------------------------
glasgow1_t~3 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
```

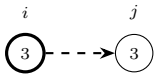
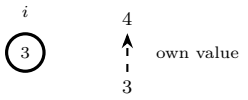
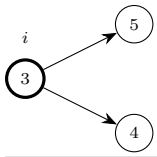
Diagram	Statistic	Description
	$\sum_j x_{ij}\,(\mathrm{sim}_{ij} - \overline{\mathrm{sim}})$ with $\mathrm{sim}_{ij} = 1 - \lvert z_i - z_j \rvert / R$	`behsim` (network side, selection): i names j whose behavior value is close to i's.
	z_i	`linear` (behavior side): i's own tendency to move to a higher value, whoever its friends are; a negative coefficient favors lower values.
	$z_i \sum_j x_{ij} z_j \,/\, \sum_j x_{ij}$	`avalt` (behavior side, influence): i moves toward the average value of the alters it names (here 4.5).

Figure 4: What the effects of the co-evolution model count. The numbers in the circles are values z of the behavior (alcohol use), and R is its range. In the `linear` row there is no tie: the dashed arrow is a change in i's own value (here from 3 to 4), which is what an actor considers when it has a turn to change its behavior, and the statistic depends on i's own value only. Statistics are shown without the centering that RSiena applies to z and to similarity.

outdegree	-2.384162	.1104054	-21.59	0.000	-2.600553	-2.167772
reciprocity	2.768611	.1877528	14.75	0.000	2.400622	3.1366
behsim	1.190873	.488049	2.44	0.015	.2343144	2.147431
beh_linear	.3529297	.1294035	2.73	0.006	.0993036	.6065558
beh_avalt	.2532406	.2030569	1.25	0.212	-.1447437	.6512249

```
Rate parameters (estimated, one per inter-wave period):
```

	period1	period2
network_rate	5.8324	4.5480
se	0.9548	0.7259
behavior_r~e	1.2297	1.8179
se	0.3183	0.4727

```
Overall maximum convergence ratio: 0.089 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

Pupils with similar alcohol use are more likely to become friends (`behsim`, $p = 0.015$), while there is no evidence that pupils' alcohol use moves toward that of their friends (`beh_avalt`, $p = 0.212$). On this panel, there is evidence for selection and none for influence. A co-evolution model has two rates per period, one for the network and one for the behavior, and both are estimated with the other parameters (unconditional estimation, as in RSiena). The fit used 2,000 replicates in Phase 3 and meets both convergence criteria (overall ratio 0.089, every t-ratio below 0.1). The estimates, standard errors and rates agree with RSiena's (Section 5.7). `estat gof` adds the distribution of the behavior after a co-evolution model:

```
. estat gof, seed(12345) name(glasgow_coev_gof_plot)
Goodness of fit: Mahalanobis distance test, 50 simulations at the estimates (2 p
> eriods pooled)
```

Statistic	Mahalanobis	P-value	Fit
Outdegree	7.080	0.440	ok
Indegree	12.434	0.140	ok
Geodesic	215.918	0.000	rejected
Behavior	7.873	0.040	rejected

```
Fit is rejected when P < 0.05 (one-tailed test).
Violin plots: glasgow_coev_gof_plot_outdegree glasgow_coev_gof_plot_indegree gla
> sgow_coev_gof_plot_geodesic glasgow_coev_gof_plot_behavior
```

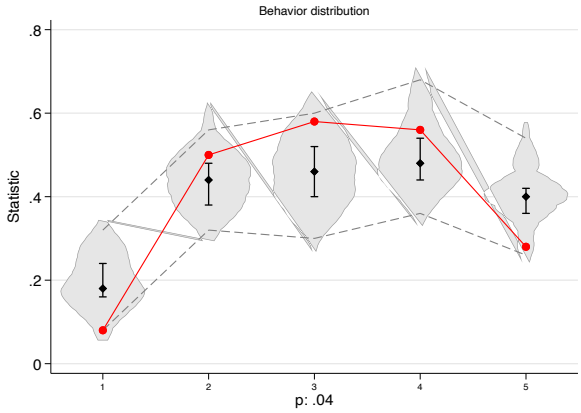


Figure 5: `estat gof`'s behavior-distribution violin plot for the co-evolution model above.

Outdegree and indegree show no evidence against the fit ($p = 0.440$, $p = 0.140$). Geodesic distance again does ($p < 0.02$). The behavior distribution is borderline: it is rejected with 50 simulations ($p = 0.040$) but not with 1,000 ($p = 0.061$; RSiena's `sienaGOF`: $p = 0.077$), so the model may not fully reproduce how alcohol use is distributed at the later waves, and adding behavior effects such as `quadratic` would be a natural check.

5.6 Endowment and creation functions

Every effect discussed so far treats a step up and a step down (in the behavior) or creating and dropping a tie (in the network) symmetrically. Endowment and creation effects split an effect into two, so that, for example, the tendency to reduce alcohol use can differ from the tendency to increase it. `nwsaom` offers the split for `quadratic`, `avalt`, and `avsim` on the behavior side, and for `reciprocity` on the network side. The example splits the quadratic shape effect:

```
. nwsaom, waves(glasgow1 glasgow2) outdegree reciprocity behavior(alcohol1 alcoh
> ol2) linear quadraticendow quadraticcreation k0(50) k3(1000) seed(12345)

SAOM co-evolution (Method of Moments), waves: glasgow1 glasgow2
Actors: 50                             Periods: 1
Behavior: alcohol1 alcohol2
```

glasgow1_to~2	Coefficient	Std. err.	z	P>|z|	[95% conf.	interval]
outdegree	-2.230546	.1230302	-18.13	0.000	-2.471681	-1.989411
reciprocity	2.440944	.2424068	10.07	0.000	1.965835	2.916052
beh_linear	.4112283	.3248725	1.27	0.206	-.2255102	1.047967
beh_quadrat~w	.0843315	.4764377	0.18	0.860	-.8494692	1.018132
beh_quadrat~n	-.1962537	.6170926	-0.32	0.750	-1.405733	1.013226

```
Rate parameters (estimated, one per inter-wave period):
```

```
------------------------------------
                |          period1
----------------+-------------------
network_rate    |           5.4855
          se    |           0.8234
behavior_r~e    |           1.2206
          se    |           0.3590
------------------------------------
Overall maximum convergence ratio: 0.170 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): not met for r
> ate_period1 (-0.131)
Convergence is good only if both criteria are met. Refit starting from these est
> imates with theta0(); see nwsaom.
```

Neither part is significant here ($p = 0.860$ and 0.750), which is common: separate endowment and creation effects need many changes in both directions, and RSiena's manual warns that they are harder to estimate than the symmetric effect. The overall ratio (0.170) meets its criterion, but the t-ratio of the network rate (-0.131) does not, and the output advises a refit from these estimates with `theta0()`, as in Section 5.3. The linear shape effect and `outdegree` cannot be split: their endowment and creation statistics add up to the statistic of the rate, so with the rate estimated the model is not identified, and `nwsaom` refuses it with that explanation.

5.7 Checking the examples against RSiena

The models of this section were also fitted in RSiena 1.6.14 (R 4.6.0) on the same data, with the same effects and 1,000 Phase 3 simulations, five seeds each. Table 3 shows two of them. Over all coefficients of the four models fitted with several seeds (the first fit, the model of Section 5.3, the co-evolution model and the endowment and creation model) the mean estimates of the two programs differ by at most 0.04 RSiena standard errors, the ratio of the standard errors lies between 0.97 and 1.05, and the network rates agree to within 0.03 standard errors. For comparison, the seed-to-seed standard deviation of an RSiena estimate is below 0.04 in every case.

5.8 Fixing effects, simulating, testing, and combining groups

`fix(`*effects*`)` keeps effects at their `theta0()` values (RSiena's `fix = TRUE`). It is used to fit a restricted model for a score-type test, and with `nwsaom simulate` to simulate at chosen coefficients without estimating anything (RSiena's `siena07` with `simOnly`); after a fit, `estat simulate` simulates the fitted model. `estat score` is the generalised score-type test of Schweinberger (2012): it tests, from the simulations of the restricted model alone, whether freeing the fixed effects would improve the fit. `estat timetest` tests whether the effects have the same size in every period (RSiena's `sienaTimeTest`), and `estat importance` computes the relative importance of the effects (RSiena's `sienaRI`, called `interpret_size` in current versions; also for non-directed and two-mode networks and for the behavior effects of co-evolution fits). All were compared with RSiena: `estat score` and `estat timetest` within simulation noise over eight seeds, `estat importance` to 10^{-7}.

First a score-type test of two effects that are not in the model. They are held fixed at 0 in a restricted fit, and `estat score` asks what freeing them would do:

Table 3: The estimates of two examples, `nwsaom` and RSiena: mean over five seeds and mean standard error (Phase 3 with 1,000 simulations in both programs)

	`nwsaom`		RSiena	
Effect	Estimate	s.e.	Estimate	s.e.
Model of Section 5, first fit				
`outdegree`	−2.804	0.165	−2.805	0.162
`reciprocity`	2.610	0.192	2.617	0.191
`nodematch(smoke1)`	0.126	0.147	0.128	0.151
`cycle3`	1.156	0.149	1.150	0.149
Co-evolution model				
`outdegree`	−2.384	0.113	−2.385	0.108
`reciprocity`	2.767	0.190	2.770	0.184
`behsim`	1.175	0.481	1.154	0.484
`linear`	0.353	0.137	0.353	0.131
`avalt`	0.252	0.205	0.246	0.196
network rate, period 1	5.867	1.013	5.891	1.011
network rate, period 2	4.533	0.693	4.519	0.683

```
. quietly nwsaom, waves(glasgow1 glasgow2) outdegree reciprocity transtrip cycle
> 3 theta0(-2.3 2 0 0) fix(transtrip cycle3) k0(100) k3(1000) seed(1)
. estat score
Score-type test of fixed effects
H0: each effect below equals its fixed value
```

Effect	Fixed at	chi2	df	Prob > chi2	One-sided z
transtrip	0.0000	81.920	1	0.0000	9.051
cycle3	0.0000	67.902	1	0.0000	8.240
Joint		82.061	2	0.0000	

```
One-step estimates (restricted estimates plus the one-step correction)
```

Effect	Estimate
outdegree	-2.6717
reciprocity	2.0383
transtrip	1.3303
cycle3	0.2607

Both effects would improve the fit; the one-step estimates show roughly what they would be if freed. With three waves, `estat timetest` asks whether the effects of a fitted model differ between the two periods:

```
. quietly nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transt
> rip k0(50) k3(1000) seed(12345)
. estat timetest
Test of time heterogeneity (score-type test)
H0: the parameter of each effect is the same in all periods
```

Effect	chi2	df	Prob > chi2

outdegree	0.102	1	0.7489
reciprocity	1.095	1	0.2953
transtrip	0.465	1	0.4953
Joint	4.170	3	0.2437

Period-specific deviations (one-step estimates) and separate tests

Effect	Period	Deviation	chi2	Prob > chi2
outdegree	2	-0.3452	0.102	0.7489
reciprocity	2	0.7343	1.095	0.2953
transtrip	2	0.0543	0.465	0.4953

and `estat importance` shows which effects matter most for the actors' choices (here for `outdegree reciprocity transtrip cycle3`, with the expected share of each effect in each period, `RI p1` and `RI p2`):

```
. quietly nwsaom, waves(glasgow1 glasgow2 glasgow3) outdegree reciprocity transt
> rip cycle3 k0(50) k3(1000) seed(12345)
. estat importance
```

Relative importance of effects
Expected over the actors at the start of each period

Effect	Estimate	Std. err.	Sigma	Semi-std.	RI p1	RI p2
outdegree	-2.6890	0.1144	0.4384	-1.1788	0.4343	0.5050
reciprocity	2.4734	0.2155	0.2140	0.5293	0.3787	0.3105
transtrip	0.6695	0.1445	0.6514	0.4361	0.1739	0.1723
cycle3	-0.0931	0.2876	0.3374	-0.0314	0.0132	0.0122
Certainty					0.2652	0.3506

RI: share of the effect in the actors´ choices. Semi-std.: estimate x sigma.
Certainty: RSiena´s R_H, the expected certainty of the choices.

Simulation at chosen coefficients needs no fit. The following makes four groups by simulating the model four times from the same start network, fits each group, and combines the four fits with `nwsaom meta`; the combined means should be close to the coefficients used (-2.3, 2, and 0.5):

```
. nwsaom simulate, waves(glasgow1 glasgow2) outdegree reciprocity transtrip
>     theta(-2.3 2 0.5) rates(5) nsim(4) seed(7) generate(g)
. forvalues s = 1/4 {
.     quietly nwsaom, waves(glasgow1 g_‘s’_p1) outdegree reciprocity transtrip k3(500) seed(‘s’)
.     estimates store grp‘s’
. }
. nwsaom meta grp1 grp2 grp3 grp4
Meta-analysis of 4 fits
Estimates with a standard error of 10 or more are excluded;
tests of the means use a t distribution with n-1 d.f.

IWLS estimates (Snijders and Baerveldt 2003)
------------------------------------------------------------------------------
      Effect |   n      Mean  Std. err.       t   P>|t|      SD       Q     P>Q
-------------+----------------------------------------------------------------
   outdegree |   4   -2.2509     0.0579  -38.87   0.000  0.1158    2.59   0.460
 reciprocity |   4    1.8694     0.0737   25.37   0.000  0.1473    1.18   0.759
```

```
   transtrip |   4    0.4758     0.0114   41.81   0.000  0.0228    0.14   0.987
-------------------------------------------------------------------------------
```

No effect differs between the groups (`P>Q` between 0.46 and 0.99), as expected for groups simulated from one model. In an application the groups would be, for example, school classes, and a small `P>Q` would say that the effect differs between classes by more than the standard errors allow.

5.9 Several groups: joint estimation and meta-analysis

Many panels consist of several separate groups: school classes, teams, villages. Each group is its own network, with its own actors, and ties never run between groups. There are two ways to analyze such data, and `nwsaom` offers both, as RSiena does. One can fit the same model to each group separately and combine the estimates (a meta-analysis, `nwsaom meta`), or fit one model to all groups at once (joint estimation, the option `groups()`; RSiena's `sienaGroupCreate()` followed by `siena07`).

Joint estimation. In a joint model all groups share *one set of effect parameters*: actors in every group are assumed to choose their ties by the same rule, with the same weight for reciprocity, transitive closure and so on. What is not shared is the speed of change: every period of every group has its own rate parameter, because classes differ in how much they change. The statistics of all periods of all groups are added up when the coefficients are estimated, just as the periods of one group are in a model with three or more waves. Small groups that are each too small for a model of their own can therefore be analyzed together, and the standard errors reflect the pooled information. The price is the assumption that the groups do not differ; the model does not estimate how much they do.

The groups are stacked in one data set. The rows are the actors of all groups, one variable (here `class`) says which group an actor belongs to, and each wave of the network is one `nwcommands` network on all actors that has no tie between two groups (a block-diagonal adjacency matrix). Covariates are ordinary variables; a tie between two groups stops the command with their number. The option `gwaves()` is for groups observed at different occasions: `gwaves(2: 1 2 | 3: 2 3)` says that group 2 was observed at the first two entries of `waves()` and group 3 at the last two. Rates are named with RSiena's period numbers, in which the step from the last wave of one group to the first wave of the next is not a period: four groups with two waves each have `period1 period3 period5 period7`. Everything that RSiena keeps per group is per group here as well: the similarity mean of a covariate, the balance mean, the mean and variance of a behavior and the centering of a dyadic covariate. An actor covariate is centered at its mean over all actors.

The example uses four Dutch school classes of 45, 37, 33 and 36 pupils, observed twice, from the Dutch Social Behavior Data Set of Chris Baerveldt that ships with RSiena (Snijders and Baerveldt, 2003). The stacked networks are `w1` and `w2`, and `class` is the group variable.

```
. nwsaom describe, waves(w1 w2)

Network data: 2 waves, 151 actors, directed
```

```
------------------------------------------------------
        Wave |     ties     density   avg. degree
-------------+----------------------------------------
           1 |      246      0.0109         1.629
           2 |      335      0.0148         2.219
------------------------------------------------------
. nwsaom, waves(w1 w2) groups(class) outdegree reciprocity transtrip seed(5)
SAOM (Method of Moments, conditional), waves: w1 w2
Actors: 151                           Periods: 4
Groups: 4 (variable class), actors per group: 45, 37, 33, 36; waves per group: 2, 2, 2, 2
------------------------------------------------------------------------------
    w1_to_w2 | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
-------------+----------------------------------------------------------------
   outdegree |  -2.246167   .0820687   -27.37   0.000    -2.407018   -2.085315
 reciprocity |   1.862864   .1569308    11.87   0.000     1.555285    2.170443
   transtrip |   .4655751   .0541992     8.59   0.000     .3593466    .5718035
------------------------------------------------------------------------------
Rate parameters (estimated, one per inter-wave period):
------------------------------------------------------
             |   period1    period3    period5    period7
-------------+----------------------------------------
        rate |    6.4989     5.3821     2.2893     8.2490
          se |    0.9714     1.4169     0.5299     2.1306
------------------------------------------------------
Overall maximum convergence ratio: 0.017 (criterion: below 0.25; met)
Convergence t-ratios (criterion: all below 0.1 in absolute value): met
```

There is one coefficient table, because the weights of the three effects are common to all 151 pupils: ties are reciprocated and closed beyond what density alone gives. There are four rates. Pupils in class 4 (rate 8.2) had about three and a half times as many opportunities to change a tie as pupils in class 3 (2.3). The matrices `e(groupn)`, `e(rates_group)` and `e(rates_period)` say which rate belongs to which group and to which of RSiena's period numbers. The same calls also work for one or several co-evolving behaviors in all groups (the shared effects, one network rate and one rate for each behavior per period, and for each behavior the mean and the similarity mean of the group), for groups with different numbers of waves, and, after the fit, for `estat timetest` (joint, per effect, per group and per period) and `estat score`. `estat gof`, `estat simulate`, `nwsaom simulate` and `estat importance` work per group (the option `group(#)`, as RSiena's `sienaGOF(groupName = )`), and `estat gof` combines the p-values of the groups by the inverse-normal method.

Meta-analysis. `nwsaom meta` takes the stored estimates (`estimates store`) of the same model fitted to each group, with the same effects in the same order. For every effect it reports the combined mean and its standard error (an iteratively reweighted least squares estimate with a random effect for the group, Snijders and Baerveldt, 2003), the estimated standard deviation of the effect between groups, a heterogeneity test Q, maximum-likelihood estimates of the mean and the between-group standard deviation with profile-likelihood intervals, and Fisher's combination of the one-sided tests; all agree with RSiena's `siena08` to five decimals. Here are the four classes, fitted separately with the same model, and combined:

```
. forvalues s = 1/4 {
.     quietly nwsaom, waves(school`s'_w1 school`s'_w2) outdegree reciprocity
>         transtrip seed(5)
.     estimates store school`s'
. }
```

```
. nwsaom meta school1 school2 school3 school4
Meta-analysis of 4 fits
Estimates with a standard error of 10 or more are excluded;
tests of the means use a t distribution with n-1 d.f.

IWLS estimates (Snijders and Baerveldt 2003)
------------------------------------------------------------------------------
      Effect |   n      Mean  Std. err.       t   P>|t|      SD       Q     P>Q
-------------+----------------------------------------------------------------
   outdegree |   4   -2.3922     0.2552   -9.37   0.003  0.5104    9.44   0.024
 reciprocity |   4    1.8994     0.2698    7.04   0.006  0.5397    6.51   0.089
   transtrip |   4    0.6963     0.1711    4.07   0.027  0.3422    8.53   0.036
------------------------------------------------------------------------------

Maximum likelihood under normality: mean of the parameter
------------------------------------------------------------
      Effect |      Mean  Std. err.   [95% conf. interval]
-------------+----------------------------------------------
   outdegree |   -2.2584     0.1071      -2.8379   -1.9836
 reciprocity |    1.8673     0.2259       1.3384    2.5535
   transtrip |    0.5202     0.0547       0.3965    1.0588
------------------------------------------------------------

Maximum likelihood under normality: standard deviation between groups
-------------------------------------------------
      Effect |        SD    [95% conf. interval]
-------------+-----------------------------------
   outdegree |    0.1163        0.0000    0.8546
 reciprocity |    0.2797        0.0000    1.0987
   transtrip |    0.0001        0.0000    0.6287
-------------------------------------------------

Fisher's combination of one-sided tests (chi-squared, 2n d.f.)
------------------------------------------------------------
             |           Right-sided            Left-sided
      Effect |     chi2  Prob > chi2     chi2  Prob > chi2
-------------+----------------------------------------------
   outdegree |    0.000        1.000  759.535        0.000
 reciprocity |  148.298        0.000    0.001        1.000
   transtrip |  118.941        0.000    0.000        1.000
------------------------------------------------------------
Correlations of estimates and standard errors are in r(meta).
```

The separate fits agree in sign and differ in size: the outdegree effect is -2.01 to -2.34 in classes 4, 1 and 2 but -3.33 in class 3, and reciprocity is 2.61 in class 2 against 1.55 to 1.57 in classes 1 and 4. The random-effects mean of reciprocity is 1.90 with a between-class standard deviation of 0.54. The heterogeneity test Q rejects equal effects for outdegree ($p = 0.024$) and transitive triplets ($p = 0.036$). The ML intervals for the between-class standard deviations start at zero, which is what four groups can tell about a variance. Fisher's test combines the one-sided tests of the four classes; a small "right" p-value speaks for a positive effect (reciprocity and closure), a small "left" p-value for a negative one (outdegree). Because only four classes enter, the tests of the means use a t distribution with 3 degrees of freedom. The option `rates` combines the rate parameters as well.

Joint versus meta. The joint estimates are -2.25 for outdegree, 1.86 for reciprocity and 0.47 for transitive triplets (Table 4 also gives RSiena's), against the random-effects means -2.39, 1.90 and 0.70 and the simple averages of the four class estimates -2.49, 2.00 and 0.79. The standard errors of the joint model (0.08, 0.16, 0.05) are much smaller

than those of the random-effects means (0.26, 0.27, 0.17), because the joint model takes the sameness of the classes for granted, while the meta-analysis lets the effects vary and estimates by how much. When Q is clearly significant, as for outdegree and transitive triplets here, the joint standard errors are too optimistic. The joint model is the first thing to try; the meta-analysis checks it; and the joint model can then be extended by letting selected effects differ between groups, by interacting them with a group indicator. Letting reciprocity differ for class 2 (`egox(c2)` with `interact(egox_c2#reciprocity)`) gives an interaction coefficient of 1.10 (standard error 0.42): pupils in class 2 reciprocate more than the others (this fit narrowly misses the t-ratio criterion, reciprocity −0.115, and a refit with `theta0()` is advised).

Table 4: Joint estimation of the four classes (151 pupils): `nwsaom` and RSiena 1.6.14. Means over ten runs of each program with seeds 1 to 10; standard errors are the means over the runs.

	Estimate		Standard error	
Parameter	`nwsaom`	RSiena	`nwsaom`	RSiena
outdegree	−2.246	−2.248	0.082	0.080
reciprocity	1.866	1.870	0.156	0.153
transtrip	0.465	0.464	0.055	0.054
rate, class 1	6.50	6.51	1.01	1.01
rate, class 2	5.37	5.38	1.38	1.16
rate, class 3	2.30	2.31	0.54	0.73
rate, class 4	8.16	8.16	2.01	1.42

Agreement with RSiena. Table 4 shows the joint model of the four classes fitted by both programs: the largest difference between the programs in an effect is 0.004, and the rates agree within 0.01. Four further joint models were compared in the same way, ten seeds each. A co-evolution model of friendship and alcohol use in three groups of 15, 20 and 15 pupils (the Glasgow excerpt cut into artificial groups for the purpose; ties between the groups removed) gives, for example, reciprocity 2.935 against 2.936 and the linear shape of alcohol use 0.386 against 0.384, with its six network and six behavior rates within 0.02 and a largest difference of an effect of 0.01. For three groups with 3, 2 and 2 waves (`gwaves()`) the estimates differ by at most 0.005 (RSiena 1.6.14 stops with an error for a group object whose groups have different numbers of waves; the reference is the equivalent object of two-wave groups). A model with two co-evolving behaviors, alcohol use and smoking, in the same three groups gives the same effects as RSiena for the network and for both behaviors, and all eighteen rates, the network rate and the two behavior rates of the six periods, agree within the simulation error. `estat timetest` after a grouped fit with the effects of an alcohol covariate in the same three groups gives a joint test of 37.5 on 25 degrees of freedom (RSiena 38.7), and the score test of two fixed effects 0.17 on 2 degrees of freedom (RSiena 0.18).

What is not available with groups. Maximum-likelihood and Bayesian estimation, two-mode networks, `nwsaom multinet` and `multiplex`, `estat` after a fit with several behaviors, `structural()` and the structural codes, and time-varying dyadic covariates stop with a message that says so. `estat importance` is not available together with `ratecov()` or `present()`. Non-directed networks, `joinat()`/`leaveat()` (on the scale of the waves of the actor's group), `behrate()` and the effects with data constants (the centered degree effects, `avdeg`, `ownx()`) work. Groups need the compiled plugin of this release; there is no Mata fallback.

A known difference from RSiena. In one case the joint estimates of `nwsaom` do not agree with RSiena's: a model of several groups with missing behavior values and the selection effects `behegox` or `behaltx`. In a single group the two programs agree, and the cause of the difference in groups has not been found, so this combination is not claimed as validated. Such joint estimates should not be relied on; one can fit the groups one at a time and combine them with `nwsaom meta`, or use RSiena.

6 Beyond the basic model

This section describes what `nwsaom` offers beyond the single directed network of the examples in Section 5. The rule behind the syntax is the usual Stata one: a property of the data is detected from the data, an option changes how a model is specified or fitted, and a different job is a subcommand or a postestimation command.

6.1 More kinds of networks and covariates

A network declared as two-mode (`nwset, bipartite`) is recognised and fitted with the effects that RSiena offers for bipartite networks, the covariate effects, changing covariates and, with `behavior()`, a co-evolving behavior of the actors. An undirected network (undirected `nwset`, or waves that are symmetric) is fitted with RSiena's model types for non-directed relations over any number of waves, also together with a behavior. Further covariate types are changing covariates (`tvcov()`), constant and changing dyadic covariates (`dyadcov()`, `dyadx()`), covariates with missing values, and the effects of a covariate on triads, popularity and activity (`netx()`). Dyads whose value is fixed by design are marked with `structural()`. `nwsaom describe` and `nwsaom simulate` also work for two-mode networks.

6.2 Several networks and a behavior: nwsaom multinet

`nwsaom multinet` fits two networks that evolve together on the same actors, as RSiena does for two dependent network variables: each network has its own effects and rates, and the cross-network effects (`crprod`, `from`, `closure` and many more, with or without a covariate) let the state of one network affect the choices in the other. One network may be non-directed, or two-mode (detected from the data, with the cross-network effects that RSiena offers for such pairs, including the dyadic-covariate effects `toU`, `sharedToU` and `avAltU`). A co-evolving behavior can be added with `behavior()`: `beheff1()` lists the effects of the first network on the behavior, `beheff2()` those of the second, and the

behavior can be a covariate of the network and cross-network effects (`behsim`, `behegox`, `behaltx`, and *effect*`:behavior` in `cross1()`). The behavior can also depend on both networks (`beheff12()`, `beheff21()`: RSiena's `FFDeg`, `BBDeg`, `indegMixedPop` and the other double-degree effects), on covariates, and on network effects with the behavior as covariate (`netbeh:`); behavior effects can have endowment and creation parts, and missing behavior values are allowed (`missbeh()`). Starting values and fixed effects are given with `theta0()` and `fix()`, so that `estat score` and `estat timetest` work after a `multinet` fit.

```
. nwsaom multinet, net1(friend1 friend2 friend3) net2(advice1 advice2 advice3)
>     behavior(smoke1 smoke2 smoke3) beheff1(linear quadratic avalt) beheff2(avalt)
```

All of these were compared with `siena07`: the observed statistics are equal and the estimates and rates agree within simulation noise. The older command `nwsaom multiplex` is deprecated and forwards to `nwsaom multinet`. Composition change and missing ties are not available in `multinet`, a two-mode member cannot be combined with a behavior, and `estat gof`, `estat importance` and `estat simulate` are not offered after it.

6.3 How much of RSiena is there

Current versions of RSiena have renamed most functions (`siena` for `siena07`, `test_gof` for `sienaGOF`, `test_time` for `sienaTimeTest`, `interpret_size` for `sienaRI`, `meta_siena` for `siena08`); this article uses the older names, which are the ones in the literature. The effects table of RSiena 1.6.6, as `getEffects()` returns it for twelve data configurations, has 1,201 rows. The effects that RSiena 1.6.7 to 1.6.14 added or changed, such as `WXZX` and the two-mode version of `toU`, are implemented and were compared with RSiena 1.6.14, but they are not part of the counts below; the counts change with RSiena releases. A row is one RSiena short name for one variable type and context. `nwsaom` implements 1,058 of the 1,201 rows in full and one in part: `unspInt`, RSiena's user-defined interaction, is available for network effects (`interact()`) but not for behavior effects. Every implemented row has been compared with `siena07`. For 1,055 of them, and for the network part of `unspInt`, the comparison covers the observed statistic and the estimates over at least two seeds. For three it covers the observed statistic only: `avAltPop` and `totAltPop`, whose parameter sits on a ridge in RSiena as well (their contribution is the same for a step up and a step down), so that the estimates of neither program can be compared; and `sameWXClosure` with a changing covariate in a two-mode member of `nwsaom multinet`, whose estimates were not compared. Table 5 lists the remaining 142 rows ($1{,}058 + 1 + 142 = 1{,}201$).

Beyond effects, `nwsaom` has no generalised-method-of-moments estimator, no settings model and no hierarchical Bayesian model. Joint estimation of several groups (`groups()`) is limited to the method of moments for one-mode networks (Section 5.9 lists the rest), and its estimates differ from RSiena's for groups with missing behavior values and the selection effects `behegox` or `behaltx`. The time-heterogeneity test has no counterpart for the orthogonalised tests of `sienaTimeTest(condition = TRUE)` or its period-wise group tests. `estat timetest` re-estimates the rate parameters by default, which matches a score test of a fitted time dummy; its option `fixrates` holds them fixed, which matches `sienaTimeTest`.

Table 5: RSiena effects that `nwsaom` does not implement (rows of RSiena's effects table)

Group	Rows
Effects of the stochastic-differential-equation model for continuous behavior	80
Statistics of the generalised method of moments (the estimator is not implemented)	31
RSiena's settings model (no estimate to compare with in `siena07`)	12
Effects that `siena07` itself cannot evaluate or does not offer (for example the two-mode `sameXOutAct`)	15
Placeholder rows of RSiena's table	2
Combinations `nwsaom` does not offer (a two-mode member of `multinet` together with a behavior; a rate effect for a continuous behavior)	2
Not implemented, total	142

7 Computational architecture

7.1 How a fit runs

`nwsaom` builds the model as a list of effects, each with two functions: one computes the effect's statistic for a whole network (the targets of the method of moments), the other an actor's contribution to its evaluation function for each possible ministep. Covariates of `egox()`, `altx()`, `nodecov()` and `ratecov()` are centered by their means, as RSiena's `coCovar()` does, unless `nocenter` is specified. In conditional estimation, the default for network-only models as in RSiena, a simulated period runs until the network is as far from its start as the observed one, and the rate is estimated from the time that takes; in unconditional estimation (co-evolution, composition change, the option `unconditional`) a period runs for a fixed time and each rate is a method-of-moments parameter. The three phases (Appendix A) follow RSiena's defaults.

Almost all of a fit's time goes into simulating the ministep process, and `nwsaom` runs these simulations, Phase 1's derivative estimates and the conditional-estimation step in a compiled C plugin, for every model it accepts. The data are passed to the plugin once per fit; Phases 1 and 3 hand it whole batches of simulations, and Phase 2, where each step starts from the previous one, calls it once per step. The older effects also have a Mata implementation with the same results but hundreds of times slower (model N1 of Section 7.2: 51 seconds against 0.22 seconds); the newer effects and joint estimation of several groups exist only in the plugin, and `nwsaom` stops with a message if the plugin is missing or too old. `e(engine)` says which ran. The Mata libraries are built with Stata 17 and work in Stata 17, 18 and 19.

Phase 1 and Phase 3 simulations are independent and run on worker threads; Phase 2 parallelizes only over the periods of a step, because each Robbins-Monro step needs the previous estimate. `cores(#)` sets the number of threads (default: all physical cores). Each simulation draws from its own random-number stream, derived from `seed()` and its position in the algorithm, so a given seed gives identical estimates whatever the number of threads.

7.2 Benchmark against RSiena

To check both correctness and speed, `nwsaom` and RSiena 1.6.14 (R 4.6.0) were run on the same 27 models and data, with each program's default settings. 20 models (N1 to T200 in Table 6) use the 50-actor Glasgow panel of this article and networks of 2, 4 or 10 disconnected copies of it (n = 100, 200, 500); network-only models use conditional estimation, except N7. The realistic models R1–R5 are the kind of specification an applied analysis would use: R1 combines `outdegree`, `reciprocity`, `gwesp()`, transitive reciprocated triplets, indegree and outdegree popularity, `samex(smoke1)`, and `egox()`, `altx()` and `simx()` on alcohol use; R2 adds alcohol use as a co-evolving behavior; R3 is undirected; R4 and R5 repeat R1 and R2 on 200 actors. The 7 further models X1 to X7 cover two-mode networks, two networks with and without a behavior, a non-directed network next to a directed one, and newer effects (the manual gives their specifications). Both programs ran single-threaded (RSiena's default; `cores(1)`), each fit in a fresh process after an untimed warm-up fit, on one machine (Apple M5 Pro (18 cores), macOS 26.5.1). The RSiena fits are from 8 November 2026, the `nwsaom` fits from release 1.5.7 (9 November 2026), with the same seeds. Release 1.5.8, which this article describes, changed only the layout of the output tables; its estimation code, Mata library and plugins are those of 1.5.7, so the estimates and timings apply to it unchanged. Other jobs ran on the machine (median one-minute load 2.3 during the RSiena fits and 2.8 during the `nwsaom` fits, on 18 cores), which affects a single-threaded fit little.

Table 6: `nwsaom` and RSiena 1.6.14 on the same models and data, both single-threaded: largest difference in a mean estimate (in RSiena standard errors) and median seconds per fit

	Specification	n	Waves	Max diff	`nwsaom`	RSiena	Speed-up
Network							
N1	`outdegree`, `reciprocity`	50	2	0.06	0.22	0.84	3.8
N2	+ `gwesp`	50	3	0.02	0.52	1.81	3.5
N4	+ `gwesp`, `samex()`	50	3	0.04	0.60	2.24	3.8
N5	+ `gwesp`, `indegpopularity`, `outactivity`	50	3	0.08	0.77	3.02	3.9
N6	+ `gwesp`, `egox()`, `altx()`, `simx()`	50	3	0.04	0.82	3.57	4.3
N7	N2, unconditional estimation	50	3	0.02	0.63	2.48	3.9
Undirected							
U1	`outdegree`, default model (forcing)	50	2	0.02	0.09	0.33	3.5
U2	`outdegree`, `symtype(joint)`	50	2	0.06	0.09	1.03	11.5
Co-evolution							
C1	N2 + alcohol: `linear`, `quadratic`, `avalt`	50	3	0.03	0.98	4.66	4.7
C2	N2 + smoking: `linear`, `quadratic`, `avsim`	50	3	0.04	1.05	5.52	5.2

continued on the next page

Table 6 continued

	Specification	n	Waves	Max diff	`nwsaom`	RSiena	Speed-up
C3	N1 + alcohol: `linear`, `avalt`	50	2	0.03	0.45	1.84	4.1
Realistic							
R1	full network model (see text)	50	3	0.06	1.33	5.29	4.0
R2	R1 network part + alcohol co-evolution	50	3	0.03	1.64	7.35	4.5
R3	undirected: `gwesp`, `samex()`, `simx()`	50	2	0.01	0.24	0.74	3.1
R4	R1	200	3	0.06	22.2	90.1	4.1
R5	R2	200	3	0.05	23.6	109	4.6
Larger networks							
S100	N2	100	3	0.05	1.97	6.82	3.5
S200	N2	200	3	0.02	8.30	31.9	3.8
S500	N2	500	3	0.03	64.5	231	3.6
T200	C1	200	3	0.08	13.8	66.5	4.8
Two-mode							
X1	`outdegree`, `indegpopularity`, `cycle4`	40×20	3	0.10	0.85	2.75	3.3
X2	`outdegree`, `indegpopularity`, `outactivity`, `cycle4`, `outiso`, `antiiniso2`	40×30	3	0.03	0.25	0.61	2.4
Two networks							
X3	both: `outdegree`, `reciprocity`; `crprod`	50	3	0.07	1.41	3.75	2.7
X4	X3 without `crprod` + smoking: `linear`, `quadratic`, `avalt` (both networks)	50	3	0.03	1.17	8.32	7.1
X5	X4 with `avalt` on network 1 only; `behegox`, `behaltx`, `behsim`, `samewwclosure:behavior`	50	3	0.06	3.91	11.5	2.9
X6	undirected network 1 (`sym1`) + directed network 2; `crprod`	50	3	0.03	1.00	3.08	3.1
Single network							
X7	`outdegree`, `reciprocity`, `gwesp`, `samextranstrip()`, `altxoutact()`, `egoplusaltsqx()`	50	3	0.02	2.22	4.26	1.9

Medians over seven fits (seeds 1–7; five above one minute, three above three minutes) per program (RSiena 1.6.14: 8 November 2026; `nwsaom` 1.5.7: 9 November 2026; same seeds). Max diff includes the rate parameters and uses the fits that converged (overall convergence ratio < 0.25) in both programs. N7, X1, X2: unconditional estimation in both programs (`unconditional`; `cond = FALSE`); X3–X6: `nwsaom multinet` with `k0(100) k3(1000)` against RSiena `n3 = 1000, nsub = 4`. Fits that converged, RSiena / `nwsaom`, for the models with at least one that did not: C2 6/6, 6/7; R2 6/7, 7/7; R4 4/5, 7/7; R5 4/5, 6/7; X1 1/7, 7/7; X3 7/7, 5/7; X4 6/7, 6/7; X5 6/7, 7/7.

The two programs give the same estimates (Table 6 and figure 6): across all 221

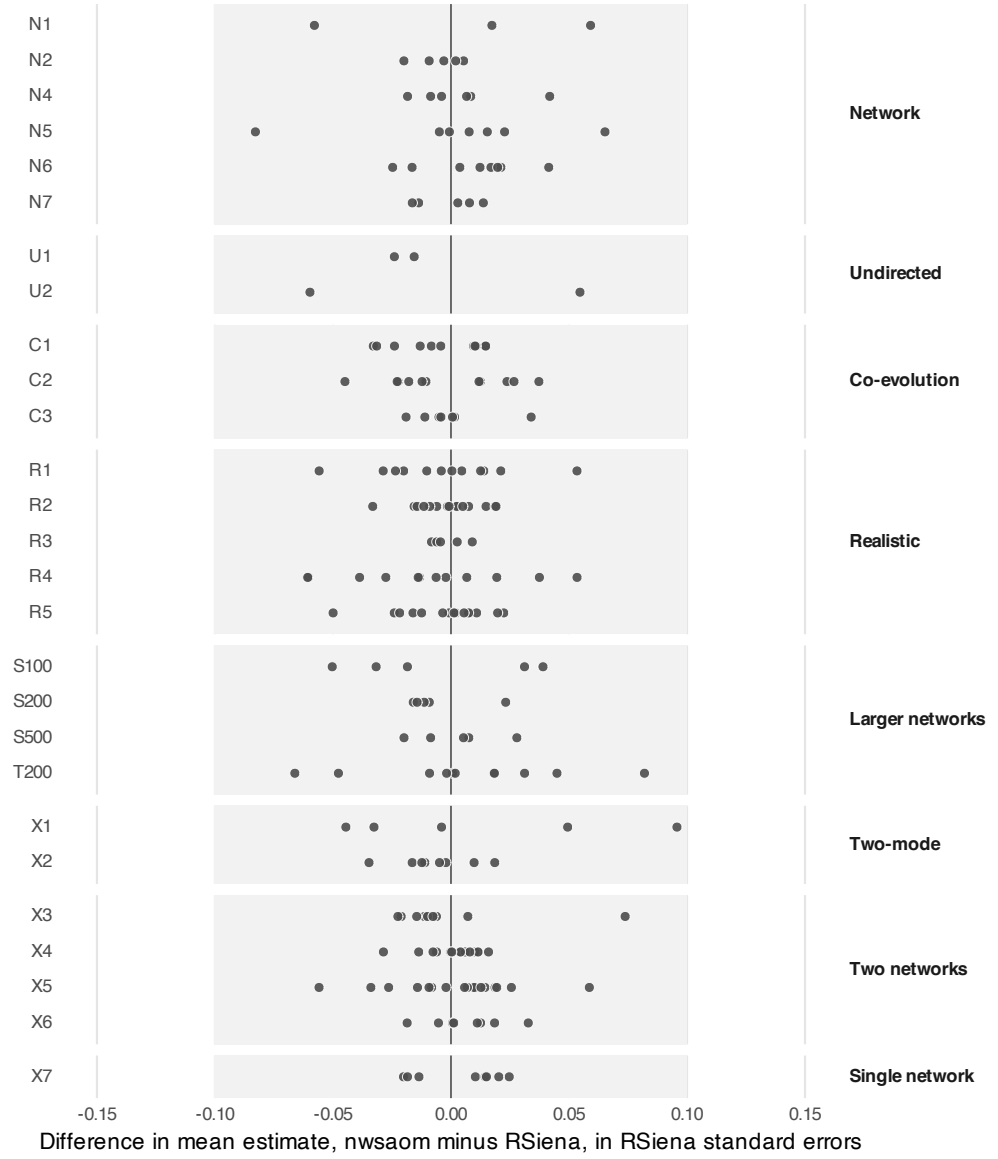


Figure 6: Difference between the mean estimates of `nwsaom` and RSiena for every parameter of every model, in RSiena standard errors. The shaded band marks ±0.1.

parameters, the mean estimates over seeds differ by at most 0.10 RSiena standard errors (on average 0.02 over all parameters), which is the size of the variation between seeds within one program (RSiena: a median of 0.03 standard errors, at most 0.11). Standard errors agree to within 7% for the network models N1 to S500; over all models the ratio of the two programs' standard errors ranges from 0.89 to 1.12, the widest in the co-evolution models, whose behavior parameters rest on less information. Of the 187 `nwsaom` fits, 182 converged by RSiena's criterion (overall maximum convergence ratio below 0.25), and of RSiena's 178 fits 167 did (six of the seven fits of X1 did not; the note to Table 6 gives the counts per model); one fit of C2 failed in RSiena and is left out. Fits that did not converge count in the timings but not in the comparison of the estimates, so for X1 the RSiena side of the comparison rests on a single fit. The comparison does not depend on this rule: with every fit included, the largest difference is 0.11 standard errors (again X1) and the mean is 0.02.

On a single thread, `nwsaom` was faster than RSiena on every model, by a median factor of 3.8 (range 1.9–11.5). The factor varies little with the effects or the network size, apart from the cheap undirected model U2 (11.5 times): the network models with `gwesp()` run 3.5 to 4.3 times as fast, the other network models 3.5 to 4.1 times, the models on 100 to 500 actors 3.5 to 4.6 times, the models with several networks 2.7 to 7.1 times, and the two-mode models X1 and X2 3.3 and 2.4 times. The fits of X1 to X7 take less than a second to twelve seconds, so their factors are less precise. In absolute terms the difference grows with the task: the realistic model on 200 actors (R4) takes 22 seconds against 90 seconds, so a model fitted a dozen times during an analysis takes about 14 minutes less.

Where the difference comes from. Both programs run the same algorithm with the same number of simulations; the difference is in how the simulation is carried out. First, RSiena runs its loop over the roughly 2,000 simulations of a fit in R and calls its compiled code once per simulation; by R's sampling profiler, the R code takes about a fifth of RSiena's time for N1 (`outdegree` and `reciprocity`) and about a tenth with `gwesp()` (N2), and more in the first fit of a new R session. `nwsaom` runs Phases 1 and 3 in batches inside the plugin, and its own loop in Phase 2 costs less than half as much per step as RSiena's R code, so most of this overhead disappears; it weighs most on the cheapest models, such as U2. Second, the plugin computes an actor's change statistics together: RSiena loops over the alternatives and, for each, over the effects (`NetworkVariable::calculateTieFlipContributions`), whereas the plugin passes once over all alternatives per effect and computes shared quantities once per actor. For example, the two-path counts between the actor and every other actor are counted in one pass and serve `gwesp()` and its variants, `gwdspff()`, `transrectrip`, `transties` and `balance`, and the geometrically weighted terms come from a table computed once per fit. That N2 is 3.5 times as fast, when removing the R overhead alone would give at most $1/0.9 \approx 1.1$, shows that the simulation itself is faster. Third, several networks share one data structure, and two-mode networks loop only over the nodes of the other mode (X2: 0.25 seconds per fit against 0.61). These explanations rest on the profile and on reading the RSiena source; RSiena's C++ code was not instrumented.

7.3 Using several cores

RSiena can spread a fit over several R processes (`siena07(..., useCluster = TRUE, nbrNodes =` k`)`); `nwsaom` uses threads by default. Table 7 fits the realistic models R1, R4 and R5 with 1, 2, 5 and 10 threads in `nwsaom` and 1, 5 and 10 R processes in RSiena (three seeds each).

Table 7: `nwsaom` with 1 to 10 threads (in parentheses, speed-up over one thread) and RSiena with 1, 5 and 10 R processes: median seconds per fit over three seeds. The estimates of `nwsaom` are identical for every number of threads.

		`nwsaom`				RSiena, processes		
Model	n	`cores(1)`	`cores(2)`	`cores(5)`	`cores(10)`	1	5	10
R1	50	1.34 (1.0)	1.11 (1.2)	0.96 (1.4)	0.92 (1.5)	4.74	3.53	3.97
R4	200	22.0 (1.0)	18.1 (1.2)	15.4 (1.4)	14.7 (1.5)	89.5	34.4	29.7
R5	200	23.6 (1.0)	14.0 (1.7)	11.7 (2.0)	10.9 (2.2)	111	46.0	49.1

Ten threads make R1 and R4 about 1.5 and 1.5 times as fast and the co-evolution model R5 2.2 times; the gain is less than proportional because Phase 2 cannot be split across threads, and most of it comes from the first few threads. Several R processes help RSiena most on the larger models (R4: 29.7 seconds with ten processes against 14.7 seconds for `nwsaom` with ten threads). With several cores in both programs, `nwsaom` is 2.0 to 4.2 times as fast as RSiena's best time on these three models.

8 Conclusions

`nwsaom` brings stochastic actor-oriented models to Stata: method-of-moments estimation with RSiena's three-phase algorithm, an effect library of 1,058 of the 1,201 effects that RSiena 1.6.6 offers, each compared with `siena07`, undirected and two-mode networks, joint network-behavior co-evolution with one or several behaviors, several co-evolving networks, and after the fit goodness of fit, simulation, score-type tests, a test for time heterogeneity, relative importance of effects, joint estimation of several groups and meta-analysis of groups. Its simulations run in a compiled, multithreaded plugin. On the 27 benchmark models of Table 6 the estimates agree with those of RSiena, and the time per fit on a single thread is lower by a median factor of 3.8 (up to 11.5 times, for the undirected pairwise model); on the further models, including the two-mode and the several-network models, the factors are 1.9 to 7.1. Several cores shorten the time further. The contribution of the article is that these models can now be estimated, checked and reported in Stata, within the effect library described here: specification search, convergence restarts, robustness checks across seeds and goodness-of-fit simulations, together with the data management and the other analyses the Stata user already has. RSiena remains the reference implementation and the program to use for what `nwsaom` does not offer.

What `nwsaom` leaves out is listed in Section 6.3: the continuous-behavior model, the generalised method of moments, the settings model and hierarchical Bayesian estimation. Joint estimation of several groups, maximum likelihood and Bayesian esti-

mation, and `nwsaom multinet` each cover a subset of the models (Sections 5.9 and 6, Appendix A), and valued and signed networks are not supported. All results in this article were produced on macOS. Rerun on Windows (Stata 19 MP, with the Windows plugin), the examples of the article gave the same numbers in every digit that Stata displays (they agree to at least 12 significant digits), and the full test suite of `nwsaom` passed there as well; the Linux plugin is built by continuous integration and was not tested on a Linux machine. `nwsaom` refuses any model it does not support with an error that says why.

A Estimation in detail

A SAOM's likelihood cannot be evaluated directly: writing it down would mean summing over every ministep sequence that could have produced the two observed waves, and there is no closed form for that sum. By default `nwsaom` therefore estimates θ by the *method of moments*, using Robbins-Monro stochastic approximation – RSiena's default, and the estimator every benchmark in this article compares against. In practice: simulate the ministep process forward from the first wave under some trial θ, then adjust θ until the simulated end-of-period statistics line up with the observed ones on average. `nwsaom` uses RSiena's three-phase procedure with RSiena's defaults: Phase 1 runs `k0()` simulations at the starting values (by default $\max(50, 7 + 3p)$ for p parameters) to estimate, via a score-function identity, how expected statistics move with θ; Phase 2 works through four Robbins-Monro subphases, each with its own iteration count and a gain that begins at `firstg()` and is halved at every subphase, corrects each simulated statistic by its regression on its score (RSiena's `dolby` option), and runs a subphase again when its steps oscillate; Phase 3 draws `k3()` replicates at the final θ to compute standard errors (a sandwich-form covariance matrix) and a convergence *t-ratio* for each parameter, rates included: the deviation of the simulated from the observed statistic, divided by its standard deviation. This checks whether the moment conditions are met, a different question from the Wald test reported alongside each coefficient. RSiena's criteria are used: every t-ratio below 0.1 in absolute value and an overall maximum convergence ratio below 0.25. Below the coefficient and rate tables, `nwsaom` prints the overall ratio and names any parameter whose t-ratio is 0.1 or more; if the fit has not converged, it says so and suggests restarting from the estimates with `theta0()`. The option `detail` lists every t-ratio, and all of them are stored in `e(tconv)`. Section 5.3 below shows how to read this summary.

The likelihood can be handled by treating the ministep sequence between two waves as missing data. A Markov chain over such sequences, with Metropolis-Hastings moves that insert, delete and permute ministeps, gives the complete-data score of the likelihood; this is the machinery behind RSiena's maximum-likelihood estimator (`maxlike = TRUE`) and behind its Bayesian estimation, which is provided by the multiSiena package (Snijders et al., 2024) (`multi_siena`, formerly `sienaBayes`), following Koskinen and Snijders (2007). `nwsaom` offers both for a single directed network, with or without one co-evolving behavior, without missing data or composition change. The option `ml` starts from the method-of-moments fit, runs a Robbins-Monro algorithm on the complete-data score, and replaces the estimates, covariance matrix and rates by the

maximum-likelihood ones, so that `estat` commands keep working. On the Glasgow data with `outdegree`, `reciprocity` and `transtrip` it gives −2.64 and −2.63, 2.22 and 2.23, and 0.69 and 0.68 over two seeds, against −2.66, −2.63 and −2.65, 2.22, 2.19 and 2.22, and 0.71, 0.70 and 0.70 from RSiena's maximum-likelihood runs (three seeds) on the same data; a co-evolution model agrees to within about 0.1 standard errors. The subcommand `nwsaom bayes` runs after a fit and samples the posterior of the fitted model with the schedule and controls of `multi_siena` for one group (flat prior for the effects, a normal prior around the starting rate for the rates). Its posterior means and standard deviations match RSiena's maximum-likelihood estimates and standard errors to within Monte Carlo error (for the same model, posterior means −2.671, 2.255 and 0.707 with standard deviations 0.108, 0.185 and 0.072), and an exact enumeration on four actors reproduces the expected number of ministeps. This checks the posterior against RSiena's maximum likelihood, which a flat prior should reproduce, and the sampler against an exact calculation; `nwsaom bayes` has not been run against `multi_siena` itself. Both are much slower than the method of moments (about half a second for the model above): the maximum-likelihood fit took under a minute, including the method-of-moments start, against about three minutes for RSiena's maximum-likelihood fit of the same model, and a default Bayesian run took under two minutes. This is why the method of moments remains the default, as in RSiena. Several groups with a hierarchical prior, and Bayesian estimation for non-directed and two-mode networks, are not implemented, and neither maximum-likelihood nor Bayesian estimation works with the joint estimation of several groups (Section 5.9).

Also see

`nwcommands` (Grund, 2026a), which includes `nwplot` for plotting any of the observed or simulated waves a SAOM fit produces; further model packages and extensions: `https://nwcommands.org`.

Acknowledgments

`nwsaom` would not exist without the Siena project. The stochastic actor-oriented model is due to Tom Snijders (Snijders, 1996, 2001), and RSiena (Ripley et al., 2024), developed by Ruth Ripley, Tom Snijders, Zsofia Boda, Andras Voros, Paulina Preciado and many contributors, is the reference implementation that `nwsaom` follows and is compared with. The source code of RSiena and of multiSiena (Snijders et al., 2024) was the specification for the Stata code. `nwsaom` is distributed under version 3 of the GNU General Public License, which RSiena (GPL-2 or GPL-3) and multiSiena (GPL-3) permit; it is not affiliated with or endorsed by the Siena project, and errors are mine. I am grateful to the developers for making their software and manual public.

Use of AI in developing the software

The software was developed with the help of an AI coding assistant (Claude Code, Anthropic), working under my direction. I specified the scope, reviewed the code, and am responsible for the package.

About the author

Thomas U. Grund is a sociologist at the Institute of Sociology, RWTH Aachen University, working on social network analysis and computational social science. He is the author of `nwcommands`, a Stata package for the analysis of network data, and of `nwsaom`, which builds on it.

B References

Grund, T. U. 2026. nwsaom: Stochastic Actor-Oriented Models in Stata. Manual. Zenodo. `https://doi.org/10.5281/zenodo.23623807`.

———. 2026a. nwcommands: Scalable and Extensible Network Analysis in Stata. Manuscript submitted to the Stata Journal.

———. 2027. *Network Analysis with Stata: Concepts, Methods, and Applications*. College Station, TX: Stata Press. Forthcoming.

Huisman, M., and T. A. B. Snijders. 2003. Statistical Analysis of Longitudinal Network Data With Changing Composition. *Sociological Methods and Research* 32(2): 253–287.

Kalish, Y. 2020. Stochastic Actor-Oriented Models for the Co-Evolution of Networks and Behavior: An Introduction and Tutorial. *Organizational Research Methods* 23(3): 511–534.

Koskinen, J. H., and T. A. B. Snijders. 2007. Bayesian Inference for Dynamic Social Network Data. *Journal of Statistical Planning and Inference* 137(12): 3930–3938.

Lospinoso, J., and T. A. B. Snijders. 2019. Goodness of Fit for Stochastic Actor-Oriented Models. *Methodological Innovations* 12(3).

McFadden, D. 1973. Conditional Logit Analysis of Qualitative Choice Behavior. In *Frontiers in Econometrics*, ed. P. Zarembka, 105–142. New York: Academic Press.

Michell, L., and P. West. 1996. Peer Pressure to Smoke: the Meaning Depends on the Method. *Health Education Research* 11(1): 39–49.

Pearson, M., C. Steglich, and T. A. B. Snijders. 2006. Homophily and Assimilation Among Sport-Active Adolescent Substance Users. *Connections* 27(1): 47–63.

Ripley, R. M., T. A. B. Snijders, Z. Boda, A. Voros, and P. Preciado. 2024. Manual for RSiena. R package documentation.

Schweinberger, M. 2012. Statistical modelling of network panel data: Goodness of fit. *British Journal of Mathematical and Statistical Psychology* 65: 263–281.

Snijders, T. A. B. 1996. Stochastic Actor-Oriented Models for Network Change. *Journal of Mathematical Sociology* 21(1-2): 149–172.

———. 2001. The Statistical Evaluation of Social Network Dynamics. *Sociological Methodology* 31(1): 361–395.

———. 2017. Stochastic Actor-Oriented Models for Network Dynamics. *Annual Review of Statistics and Its Application* 4(1): 343–363.

Snijders, T. A. B., and C. Baerveldt. 2003. A multilevel network study of the effects of delinquent behavior on friendship evolution. *Journal of Mathematical Sociology* 27: 123–151.

Snijders, T. A. B., G. G. van de Bunt, and C. E. G. Steglich. 2010. Introduction to Stochastic Actor-Based Models for Network Dynamics. *Social Networks* 32(1): 44–60.

Snijders, T. A. B., J. Koskinen, and R. M. Ripley. 2024. multiSiena: Bayesian and Multilevel Analysis of Stochastic Actor-Oriented Models. R package, version 1.2.40, `https://www.stats.ox.ac.uk/~snijders/siena/`.

Steglich, C., T. A. B. Snijders, and M. Pearson. 2010. Dynamic Networks and Behavior: Separating Selection from Influence. *Sociological Methodology* 40(1): 329–393.